

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name: Student ID: Altynbek Dinmukhamed 230103113
Hardware / CPU: Date & Section: Apple MacOS / APPLE Silicon M1 chip / 8 cores (4 performance, 4 efficiency)

❖❖ LAB PROTOCOL & AUTONOMOUS RULES OF ENGAGEMENT

1. Autonomous Session: You are expected to read, run, optimize, and document all findings independently.
2. Zero Support for Dependency Issues: If you encounter an error, verify that you installed packages inside a clean virtual environment. Compilers are NOT needed for this lab.
3. Equal Hardware Standard: Keep your laptop plugged into wall power and close background browsers, video streams, or games to prevent CPU thermal throttling and frequency capping.
4. Whiteboard Leaderboard: When you finish Task 1 and Task 3, record your CPU model and best speedup factor on the lab whiteboard.

1. Background & Environment Setup (10 Minutes)

Standard CPython executes bytecode under the Global Interpreter Lock (GIL), preventing multiple CPU threads from executing simultaneously in Python space. In this lab, we bypass the GIL using Numba's JIT compiler with its native multithreaded OpenMP/TBB work-stealing engine. Because Numba compiles numeric Python loops directly to machine code and releases the GIL inside parallel loops (prange), we achieve bare-metal native CPU scaling across both macOS (Apple Silicon M-series) and Windows/Linux x86_64 machines with zero local C++ compiler dependencies.

Execute the following in your terminal / command prompt:

```
pip install numpy numba matplotlib
```

Instructor – sufyan.mustafa@sdu.edu.kz

2. Challenge 1: The Core Speedometer & Amdahl's Law

Objective: Benchmark an embarrassingly parallel Monte Carlo simulation to approximate π across varying thread counts. You will measure execution latency, quantify parallel speedup, and observe where Amdahl's Law caps performance.

```

import time
import numpy as np
import numba
from numba import njit, prange

print(f"Hardware Threads Detected: {numba.config.NUMBA_NUM_THREADS}")

@njit(parallel=True)
def monte_carlo_pi(n_samples):
    inside_circle = 0
    for i in prange(n_samples):
        x = np.random.uniform(0.0, 1.0)
        y = np.random.uniform(0.0, 1.0)
        if x * x + y * y <= 1.0:
            inside_circle += 1
    return (4.0 * inside_circle) / n_samples

# JIT Warmup (excludes JIT compilation overhead from benchmarks) _ =
monte_carlo_pi(10_000)

SAMPLES = 120_000_000
thread_counts = [1, 2, 4, 8, numba.config.NUMBA_NUM_THREADS]
# Filter out duplicates and invalid counts
thread_counts = sorted(list(set([t for t in thread_counts if t <=
numba.config.NUMBA_NUM_THREADS])))

print(f"{'Threads':<10} | {'Time (s)':<12} | {'Speedup':<10} | {'Efficiency (%)':<15}")
print("-" * 55)

t1_baseline = None
for t in thread_counts:
    numba.set_num_threads(t)
    start = time.perf_counter()
    pi_est = monte_carlo_pi(SAMPLES)
    elapsed = time.perf_counter() - start

    if t == 1:
        t1_baseline = elapsed
        speedup = 1.0
        efficiency = 100.0
    else:

```

Instructor – sufyan.mustafa@sdu.edu.kz

```

speedup = t1_baseline / elapsed
efficiency = (speedup / t) * 100.0

print(f"{'t':<10} | {'elapsed':<12.4f} | {'speedup':<10.2f}x |
{'efficiency':<15.1f}")

```

Deliverables & Technical Questions for Challenge 1:

■ A. Record your baseline T_1 (1 thread) and your maximum multi-thread runtime $T_{\max}$ in Table 1 below.

Threads	Time (s)	Speedup	Efficiency (%)	pi
1	0.5311	1.00	100.0	3.14162
2	0.2869	1.85	92.6	3.14187
4	0.1672	3.18	79.4	3.14139
8	0.1301	4.08	51.0	3.14183

--- Для Table 1 ---

T_1 (1 thread) = 0.5311 s

$T_{\max}$ (8 threads) = 0.1301 s

Max speedup = 4.08x, efficiency = 51.0%

■ B. Why is the warm-up call `_ = monte_carlo_pi(10_000)` strictly required before recording benchmark timings?

Numba compiles the function to machine code the first time it is called. This compilation takes several hundred milliseconds, and my whole single-thread run takes only 0.53 seconds. Without the warm-up, the compilation time would be added to T_1 . This would make the baseline too slow and the speedup of the other runs look better than it really is. The warm-up call with 10,000 samples forces the compilation to happen before the benchmark, so the timings show only the real computation.

■ C. Did your parallel efficiency stay at 100% when scaling from 1 to max threads? What architectural factors (e.g. memory bus contention, hyperthreading/SMT core vs physical core limits) explain the degradation?

No, it did not. Efficiency went down from 92.6% on 2 threads to 79.4% on 4 threads and to 51.0% on 8 threads.

My processor has 4 performance cores and 4 efficiency cores. There is no hyperthreading, but the efficiency cores are slower than the performance cores. The first 4 threads run on the fast cores and reach a speedup of 3.18 times. The next 4 threads run on the slow cores and add only about 0.9 times more, so the speedup on 8 threads is 4.08 times and the efficiency falls to 51%. This explains why the efficiency dropped sharply between 4 and 8 threads.

■ D. Why does `inside_circle += 1` inside prange not corrupt the count? What implicit OpenMP reduction construct does Numba generate here?

Numba recognizes this pattern as a reduction. Each thread gets its own private copy

of `inside_circle`, which starts at zero, and it counts only inside this copy. Because the threads do not write to the same memory location, there is no race condition. When the loop finishes, Numba adds all the partial counts together to get the final result. This is the same idea as the OpenMP construct `reduction(+:inside_circle)`. Numba generates this with its own parallel mechanism, but the logic is the same as in OpenMP.

Instructor – sufyan.mustafa@sdu.edu.kz

3. Challenge 2: Load Imbalance & Dynamic Scheduling

Objective: Compute a high-resolution 2500×2500 Mandelbrot fractal. Unlike Monte Carlo, points in the complex plane require radically different amounts of compute: points inside the set iterate all the way to `max_iter`, while outer points escape in 2-3 iterations. You will investigate how non-uniform workloads impact thread utilization.

```

import time
import numpy as np
import matplotlib.pyplot as plt
from numba import njit, prange

@njit(parallel=True)
def render_mandelbrot_rows(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Row-parallel loop
    for r in prange(h):
        cy = -1.2 + (r / h) * 2.4
        for c in range(w):
            cx = -2.0 + (c / w) * 2.5
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it < max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1
            img[r, c] = it
    return img

@njit(parallel=True)
def render_mandelbrot_cols(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Column-parallel loop (swapped iteration axis)
    for c in prange(w):
        cx = -2.0 + (c / w) * 2.5
        for r in range(h):
            cy = -1.2 + (r / h) * 2.4
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it < max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1

```

```

img[r, c] = it
return img

# Warmup JIT
_ = render_mandelbrot_rows(100, 100, 50)
_ = render_mandelbrot_cols(100, 100, 50)

H, W, MAX_IT = 2500, 2500, 1000

t0 = time.perf_counter()
grid_rows = render_mandelbrot_rows(H, W, MAX_IT)
t_rows = time.perf_counter() - t0

t1 = time.perf_counter()
grid_cols = render_mandelbrot_cols(H, W, MAX_IT)
t_cols = time.perf_counter() - t1

print(f"Row-Parallel Render Time: {t_rows:.3f} s")
print(f"Column-Parallel Render Time: {t_cols:.3f} s")

# Save visual artifact
plt.figure(figsize=(8, 8))
plt.imshow(grid_rows, cmap='magma', extent=[-2.0, 0.5, -1.2, 1.2]) plt.title(f"Mandelbrot
{H}x{W} (Render: {t_rows:.2f}s)")
plt.axis('off')
plt.savefig('mandelbrot_output.png', dpi=300, bbox_inches='tight') print("Saved image:
mandelbrot_output.png")

```

Deliverables & Technical Questions for Challenge 2:

Numba version: 0.61.0

Threads: 8

Row-Parallel Render Time: 1.921 s

Column-Parallel Render Time: 1.879 s

Results identical: True

Row-Parallel, chunksize=1 : 2.412 s

Row-Parallel, chunksize=8 : 1.200 s

Row-Parallel, chunksize=64 : 1.102 s

Saved image: mandelbrot_output.png

Benchmark	Threads	Size	Time (s)	Speedup	Efficiency
Mandelbrot Rows	Max (8)	2500 x 2500	1.921	N/A	N/A
Mandelbrot Cols	Max (8)	2500 x 2500	1.879	N/A	N/A

Rows, chunksize	Time (s)	Относительно default
default	1.921	1.00x
1	2.412	0.80x, slower
8	1.200	1.60x
64	1.102	1.74x

🖥️ A. Compare `t_rows` vs `t_cols` . Which axis decomposition completed faster on your machine and why?

The column version was slightly faster, 1.879 seconds against 1.921 seconds for rows. The difference is only about 2%, so I think it is within measurement noise and both versions have practically the same speed. This happens because the Mandelbrot task is compute-bound. Each pixel needs up to 1000 loop iterations, but the result is written to memory only one time. So the memory access pattern has almost no influence on the total time. Both versions also have a similar load imbalance, because the heavy pixels are in the middle of the image for both axes.

🖥️ B. Explain how NumPy's C-contiguous (row-major) memory layout interacts with CPU cache lines when threads write to `img[r, c]` vs `img[c, r]` .

NumPy stores arrays in C-contiguous order, so `img[r, c]` and `img[r, c+1]` are neighbours in memory. When a thread writes along a row, it fills one cache line completely before it moves to the next one. This is efficient. When a thread goes down a column, the next element is 2500 times 4 bytes, so 10,000 bytes, away. Every write touches a different cache line, and the processor must load a whole line to change only 4 bytes. False sharing can also happen at the borders between two thread blocks, when two threads write into the same cache line. In my test the difference was very small, because the writes are rare compared to the heavy calculation in the while loop. For a task with less computation per element, the row version would be clearly better.

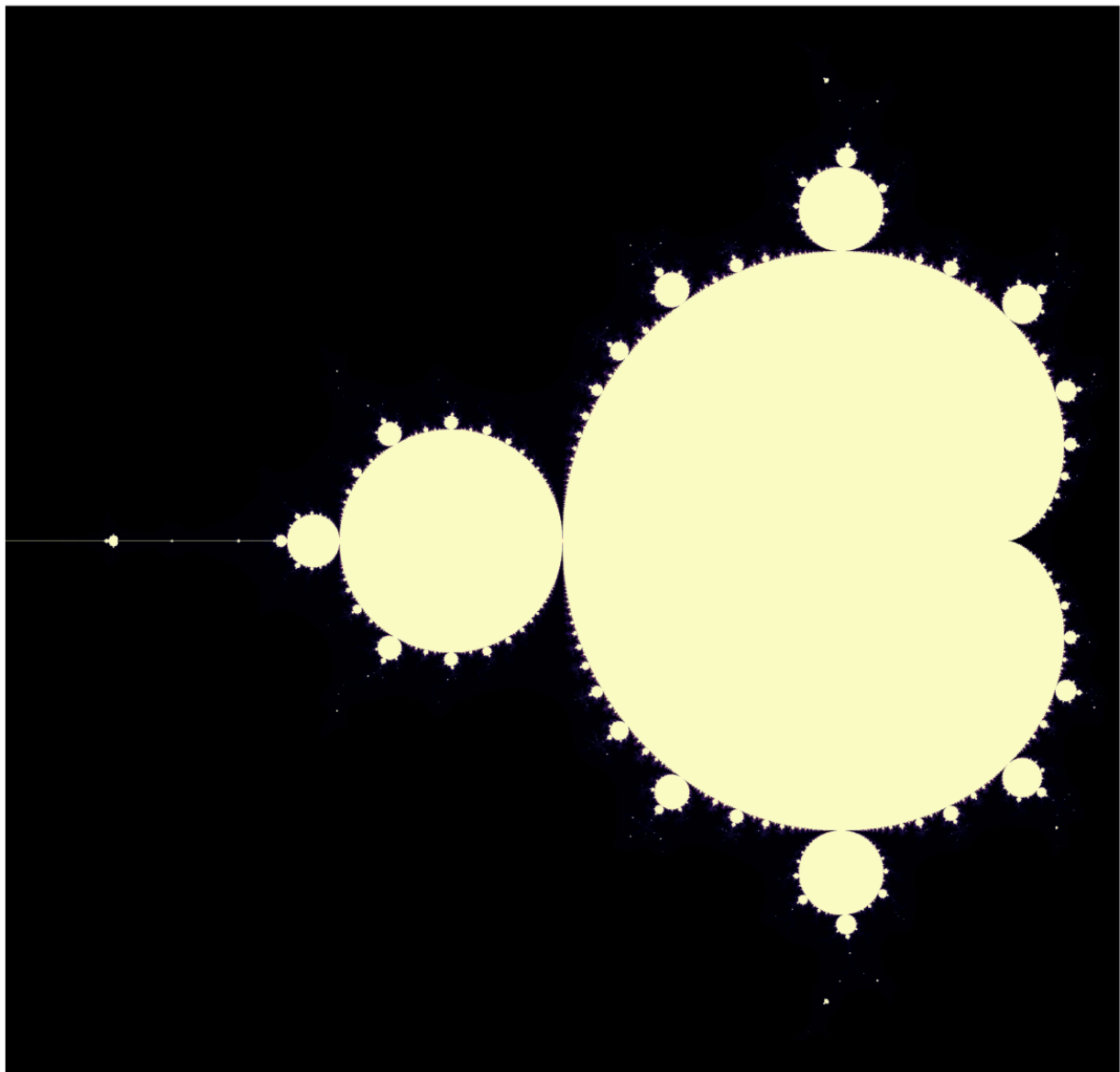
🖥️ C. Load imbalance analysis: If thread 0 receives the top 20% of rows and thread 2 receives the center 20% of rows, why does one thread idle while the other struggles? How does OpenMP dynamic scheduling resolve this?

The top rows of the image are far from the Mandelbrot set, so their points escape after

two or three iterations. Thread 0 finishes this work almost immediately and then waits. The central rows cross the set, and their points run until `max_iter`, which is 1000. Thread 2 has to do hundreds of times more work. The whole program finishes only when the slower thread finishes. My processor has 4 fast and 4 slow cores, so a static split gives the same amount of work to all threads, and the slow cores become the bottleneck.

📎 D. Attach your exported `mandelbrot_output.png` file alongside your completed task sheet.

Mandelbrot 2500x2500 (Render: 1.92s)



Instructor – sufyan.mustafa@sdu.edu.kz

4. Challenge 3: Stencil Computation & Memory Bandwidth

Objective: Simulate 2D heat diffusion using a 5-point discrete Laplacian stencil over 300 timesteps on a 1500×1500 grid. Unlike the compute-bound fractal, stencil computations are strictly memory-bandwidth bound. You will determine whether adding CPU cores continues to speed up performance or hits the RAM memory bus wall.

```

import time
import numpy as np
from numba import njit, prange

@njit(parallel=True)
def heat_step(u, u_next, alpha=0.20):
    rows, cols = u.shape
    for i in prange(1, rows - 1):
        for j in prange(1, cols - 1):
            u_next[i, j] = u[i, j] + alpha * (
                u[i+1, j] + u[i-1, j] + u[i, j+1] + u[i, j-1] - 4.0 * u[i, j] )

GRID_SIZE = 1500
STEPS = 300

u = np.zeros((GRID_SIZE, GRID_SIZE), dtype=np.float64)
u_next = np.zeros_like(u)

# Dirichlet Boundary conditions: Top & Left walls maintained at 100°C u[0, :] = 100.0
u[:, 0] = 100.0
u_next[0, :] = 100.0
u_next[:, 0] = 100.0

# Warmup
heat_step(u, u_next)

# Benchmark Execution
start = time.perf_counter()
for step in range(STEPS):
    heat_step(u, u_next)
# Pointer swap (avoids array allocation copies)
u, u_next = u_next, u
elapsed = time.perf_counter() - start

cells_per_sec = (GRID_SIZE * GRID_SIZE * STEPS) / elapsed / 1e6
print(f"Heat Diffusion Complete: {elapsed:.3f} s")
print(f"Throughput: {cells_per_sec:.2f} Megacells/sec")

```

Instructor – sufyan.mustafa@sdu.edu.kz

Deliverables & Technical Questions for Challenge 3:

Numba version: 0.61.0, threads: 8

=== float64 ===

Threads | Time (s) | Mcells/s | Speedup

1	0.338	1999.0	1.00	x
2	0.251	2687.1	1.34	x
4	0.235	2867.4	1.43	x

```

8 | 0.223 | 3022.8 | 1.51 x
=== float32 ===
Threads | Time (s) | Mcells/s | Speedup
-----
1 | 0.240 | 2815.7 | 1.00 x
2 | 0.128 | 5286.9 | 1.88 x
4 | 0.080 | 8465.6 | 3.01 x
8 | 0.083 | 8101.1 | 2.88 x
--- Для Table 1 (float64, max threads) ---
Heat Diffusion Complete: 0.223 s
Throughput: 3022.77 Megacells/sec
--- Для вопроса B ---
float32 / float64 runtime factor: 2.68x faster

```

Benchmark	Threads	Size	Time (s)	Speedup	Efficiency
Heat Stencil	Max (8)	1500 x 1500 x 300	0.223	N/A	N/A

Threads	float64 speedup	float32 speedup
2	1.34x	1.88x
4	1.43x	3.01x
8	1.51x	2.88x

🔧 A. Record your benchmark runtime and Megacells/sec throughput in Table 1. 🔧 B. Change the array precision from `np.float64` to `np.float32`. Re-run the test. By what factor did runtime decrease? Why does halving memory footprint improve stencil throughput far more than compute capability?

A. Benchmark runtime and throughput. With the float64 version on 8 threads, the heat diffusion took 0.223 seconds and the throughput was 3022.77 Megacells per second. With one thread the same run took 0.338 seconds, which is 1999 Megacells per second. I used the best time of three runs.

B. Float32 instead of float64. On 8 threads the runtime went down from 0.223 seconds to 0.083 seconds, so the float32 version was 2.68 times faster. The stencil does very little calculation for each

cell, fewer than ten simple operations, but it must move data between the memory and the processor all the time. The speed is limited by how many bytes the memory can deliver per second, not by how fast the cores can calculate. Float32 uses 4 bytes per number instead of 8, so the program moves half as much data, and the runtime falls a lot. The calculation speed of the cores did not change at all.

■ C. Memory Roofline Model: Explain why adding twice as many CPU cores does not produce a 2x speedup on stencil codes once memory bandwidth is saturated.

The roofline model says that performance is limited by the smaller of two values: the peak calculation speed of the processor, and the memory bandwidth multiplied by the number of operations done per byte. The stencil does well below one operation per byte, so it always hits the memory limit first. The memory bandwidth is shared by all cores. When it is full, new cores cannot get data faster, so they just wait.

My results show this clearly. With float64, two threads gave a speedup of 1.34 times. Going from 2 to 8 threads, which is four times more cores, increased the throughput by only about 12 percent. The memory was already saturated at two threads. With float32 each cell needs half the bytes, so the memory can feed more cores. The speedup reached 3.01 times on 4 threads, and then it stopped growing, because the memory bandwidth became the limit again. This is very different from the Monte Carlo test in Challenge 1, where the threads do not use memory and the speedup was much closer to ideal.

Instructor – sufyan.mustafa@sdu.edu.kz

5. Student Result Scorecard & Data Collection

Fill in your benchmark results into Table 1 below before submitting this document:

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
---------------------	-------------------	-------------------------	-------------------	-------------------	-------------------

Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	0.5311	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	0.2869	1.85x	92.6%
Challenge 1: Monte Carlo	4 Threads	120,000,000	0.1672	3.18x	79.4%
Challenge 1: Monte Carlo	Max Threads (8)	120,000,000	0.1301	4.08x	51.0%
Challenge 2: Mandelbrot (Rows)	Max Threads (8)	2500 x 2500	1.921	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads (8)	2500 x 2500	1.879	N/A	N/A
Challenge 3: Heat Stencil	Max Threads (8)	1500 x 1500 x 300	0.223	N/A	N/A

Instructor – sufyan.mustafa@sdu.edu.kz